

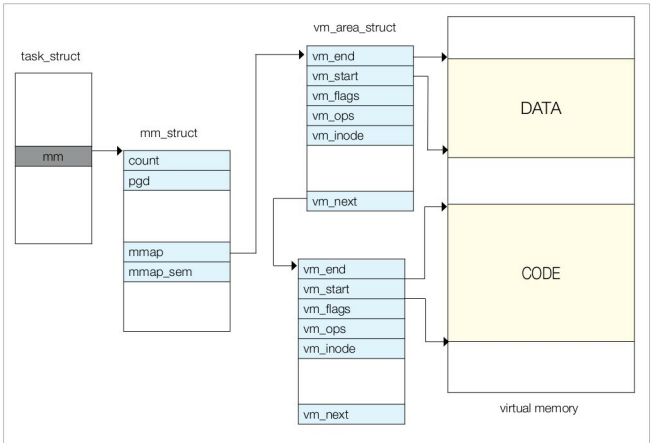


Figure 1: Virtual memory

has a pointer to the `thread_info` structure, which points to the `task_struct` structure.

Process virtual memory

Every process has its executable code and data, which is contained by the virtual memory of the process. A process also has libraries which are shared between various processes; however, the kernel does not provide a copy of the shared library to the process but gives a link to that shared library.

The kernel does not copy all code into memory but loads it on demand, i.e., demand paging. In this concept, virtual memory is loaded into the physical memory only when the process attempts to access virtual memory. Initially, all pages in the page table are marked as available in virtual memory but not in physical memory.

When a process attempts to access virtual memory, the processor tries to decode the physical address but, at that time, the page entry for that virtual address is invalid with respect to the physical address. This is because it is not loaded into the physical memory yet — which is why it raises a page fault. Now it's the responsibility of the kernel to create the physical page of the memory for that code or data, and bring this code and data from the swap area or file system into physical memory, and correct or update the 'page table entry'.



Note: Bringing the page from the virtual memory (swap area or hard disk) is a time consuming process. A context switch will take place and another process will be loaded into the processor.

`task_struct` has a pointer `mm`, which points to the `mm_struct` structure. `mm_struct` is used to maintain all the virtual memory of the process; it has a pointer `mmap` that points to the `vm_area_struct` structure, which represents the virtual memory area of the process. `vm_area_struct` has a pointer to `vm_ops` that has a set of virtual memory handling routines, which are used for various operations on the virtual memory of the process. If the process tries to access virtual memory and if it is not valid, then a page fault will occur and to handle this, one of the routines will be called by the kernel. `vm_area_struct` also has a pointer `vm_next`, which points to the next `vm_area_struct` data structure. `vm_area_struct` data structures are arranged in an AVL tree (see Figure 1).

Files

Open files of processes are maintained by `fs_struct` and `files_struct` structures, which are pointed by `fs` and `files` pointers in the `task_struct` structure, respectively. `fs_struct` contains file system information of the process. `files_struct` contains open